PROJECT SUBMISSION

Student Name: **Abhishek KR**

Register Number: **tvu35535523U18002**

Institution: Government Arts and Science College, Tirupattur

Department: **B.Sc Computer Science**

Date of Submission:**21.01.2026**

GitHub Repository Link:

https://github.com/abhi0626-kr/-Movie-Recommendation-System

Deployment URL:

https://moiverecommendationsystem.streamlit.app/

Google COLAB URL:

https://colab.research.google.com/drive/1rWAvQVHwP-y5Q5qHg1UETEjOprNIh400#scrollTo=Hqc3kA6WgZbg

1. Problem Statement

With the rapid growth of online streaming platforms such as Netflix, Amazon Prime, and Disney+, users are exposed to thousands of movie choices. While this abundance provides variety, it also creates a significant challenge in discovering movies that align with individual user preferences. Users often spend excessive time browsing rather than watching content, which negatively impacts user satisfaction and engagement.

This project addresses this problem by developing a Movie Recommendation System using content-based filtering techniques. The system analyzes movie metadata such as genres, plot summaries, cast, and directors to recommend movies that are similar in content to a user-selected movie. From a business perspective, recommendation systems help platforms increase watch time, improve customer retention, and personalize user experience.

The problem falls under the domain of recommendation systems, which is a subfield of machine learning and data science. Specifically, this project focuses on content-based recommendation, where suggestions are generated based on item similarity rather than user-user interactions.

2. Abstract

This project presents the design and implementation of a content-based movie recommendation system using the TMDB 5000 Movies dataset. The objective of the system is to recommend movies that are similar in content to a movie selected by the user. The dataset contains rich metadata such as movie overviews, genres, keywords, cast, crew, and ratings.

Exploratory Data Analysis (EDA) was conducted to understand patterns in movie ratings, popularity, genre distribution, and voting behavior. Feature engineering techniques were applied to extract meaningful information from textual and categorical features. The combined features were transformed into numerical vectors using TF-IDF vectorization. Cosine similarity was then used to measure similarity between movies.

The final system was deployed using Streamlit, providing an interactive web-based interface for users. The project demonstrates how data-driven recommendation systems can enhance decision-making and improve user experience on digital platforms.

3. System Requirements

To successfully develop and run the Movie Recommendation System, certain minimum hardware and software requirements are necessary.

Hardware Requirements:

The system requires a minimum of 8 GB RAM to handle data processing and vectorization efficiently. A modern processor such as Intel i5 or AMD Ryzen 5 (or higher) is recommended to ensure smooth execution, especially during similarity computations.

Software Requirements:
The project is implemented using Python version 3.9 or above. Development and experimentation were carried out using Google Colab and Jupyter Notebook environments. For deployment, Streamlit was used to create a web-based user interface.

Libraries and Tools:
The core Python libraries used include pandas and numpy for data manipulation, matplotlib and seaborn for visualization, scikit-learn for machine learning and vectorization, and streamlit for deployment. These tools together enable efficient data analysis, modeling, and deployment.

4. Objectives

The primary objective of this project is to design and implement an effective movie recommendation system that provides meaningful and relevant suggestions to users. The system aims to analyze movie metadata and identify similarities between movies based on content.

Specific objectives include performing exploratory data analysis to uncover trends and patterns in movie ratings and genres, engineering meaningful features from raw data, and building a content-based recommendation model using TF-IDF and cosine similarity. Another key objective is to deploy the model through a user-friendly web interface, allowing users to interact with the system easily.

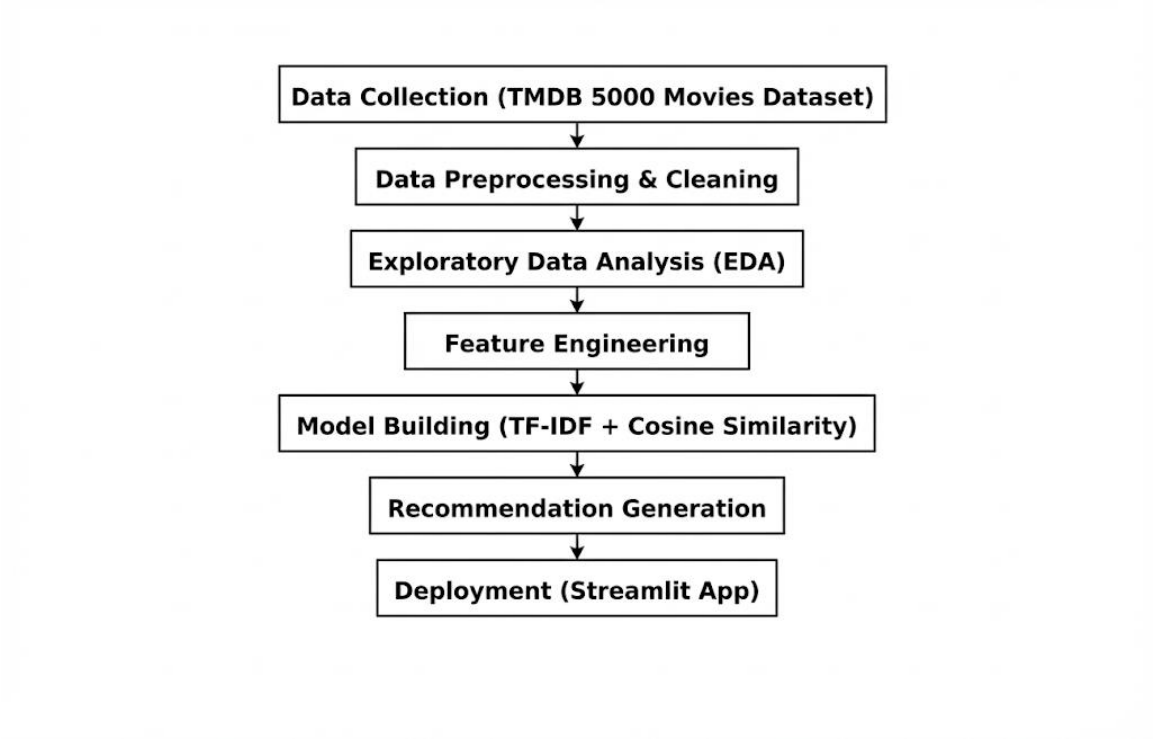
By achieving these objectives, the project demonstrates the practical application of machine learning techniques in solving real-world recommendation problems and highlights their business relevance.

5. Flowchart of Project Workflow

The workflow of the Movie Recommendation System follows a structured and systematic pipeline. The process begins with data collection from the TMDB dataset, followed by data preprocessing to clean and prepare the data for analysis. Exploratory Data Analysis (EDA) is then conducted to understand the dataset and extract insights.

Next, feature engineering techniques are applied to transform raw data into meaningful features suitable for modeling. The model building phase involves vectorizing textual features using TF-IDF and computing similarity using cosine similarity. The final stage

involves deploying the recommendation system using Streamlit.



6. Dataset Description

The dataset used in this project is the TMDB 5000 Movies Dataset, obtained from Kaggle. It is a publicly available dataset widely used for movie recommendation and analysis tasks. The dataset consists of approximately 5,000 movies and includes two main files: tmdb_5000_movies.csv and tmdb_5000_credits.csv.

The dataset contains a mixture of numerical, categorical, and textual features such as movie title, overview, genres, keywords, cast, crew, vote count, and average rating. This rich metadata makes the dataset suitable for content-based recommendation systems.

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 4803 entries, 0 to 4802

Data columns (total 20 columns):

#	Column	Non-Null Count	Dtype
0	budget	4803 non-null	int64
1	genres	4803 non-null	object

```
 2  homepage            1712 non-null   object
 3  id                  4803 non-null   int64
 4  keywords            4803 non-null   object
 5  original_language   4803 non-null   object
 6  original_title      4803 non-null   object
 7  overview            4800 non-null   object
 8  popularity          4803 non-null   float64
 9  production_companies  4803 non-null   object
10  production_countries  4803 non-null   object
11  release_date        4802 non-null   object
12  revenue             4803 non-null   int64
13  runtime             4801 non-null   float64
14  spoken_languages    4803 non-null   object
15  status              4803 non-null   object
16  tagline             3959 non-null   object
17  title               4803 non-null   object
18  vote_average        4803 non-null   float64
19  vote_count          4803 non-null   int64
dtypes: float64(3), int64(4), object(13)
memory usage: 750.6+ KB
```

	0
budget	0
genres	0
homepage	3091
id	0

	0
keywords	0
original_language	0
original_title	0
overview	3
popularity	0
production_companies	0
production_countries	0
release_date	1
revenue	0
runtime	2
spoken_languages	0
status	0
tagline	844
title	0
vote_average	0
vote_count	0

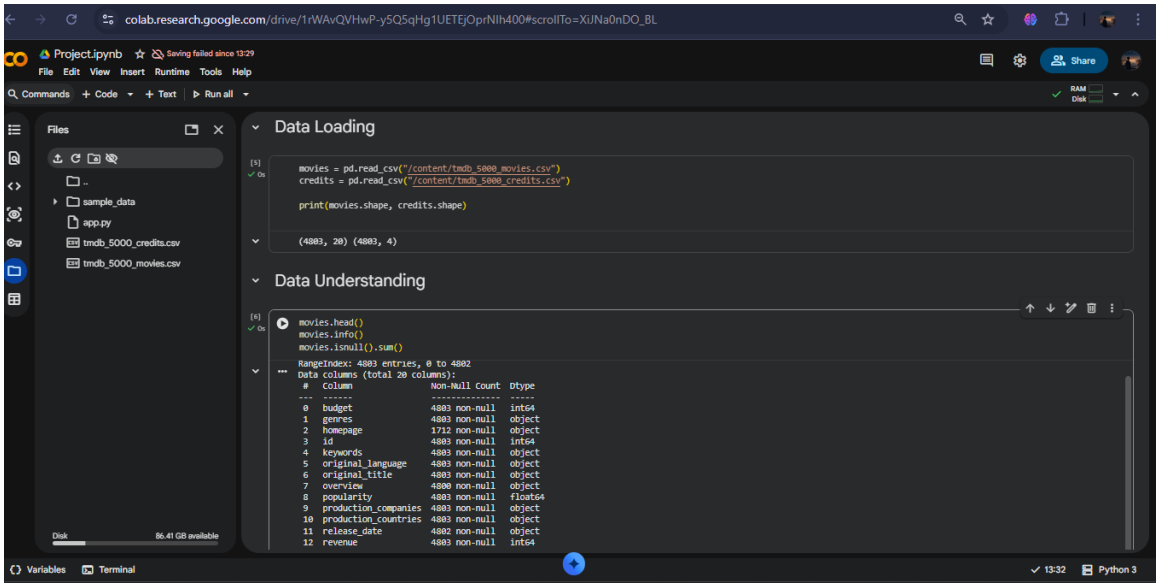
dtype: int64

7. Data Preprocessing

Data preprocessing is a critical step in ensuring the quality and reliability of the recommendation system. During this phase, missing values were identified and handled appropriately. Textual fields such as movie overviews were filled with empty strings to

prevent errors during vectorization.

Duplicate records were removed, and text normalization techniques such as lowercasing were applied. JSON-like columns were parsed to extract relevant information like genres, cast, and director. After preprocessing, the data was transformed into a clean and structured format suitable for analysis and modeling.

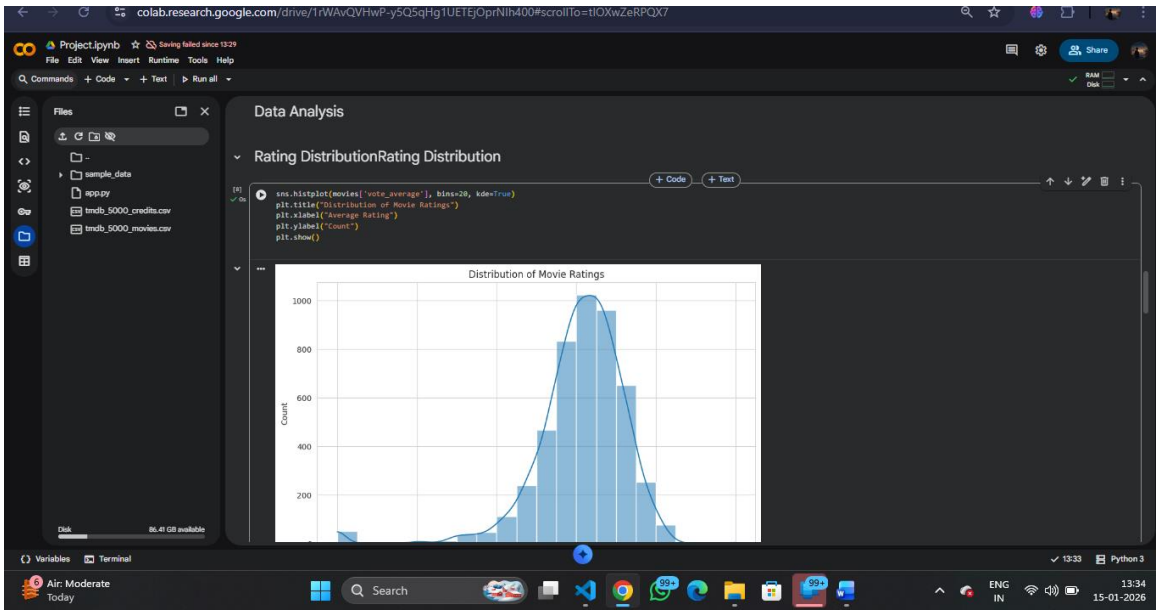


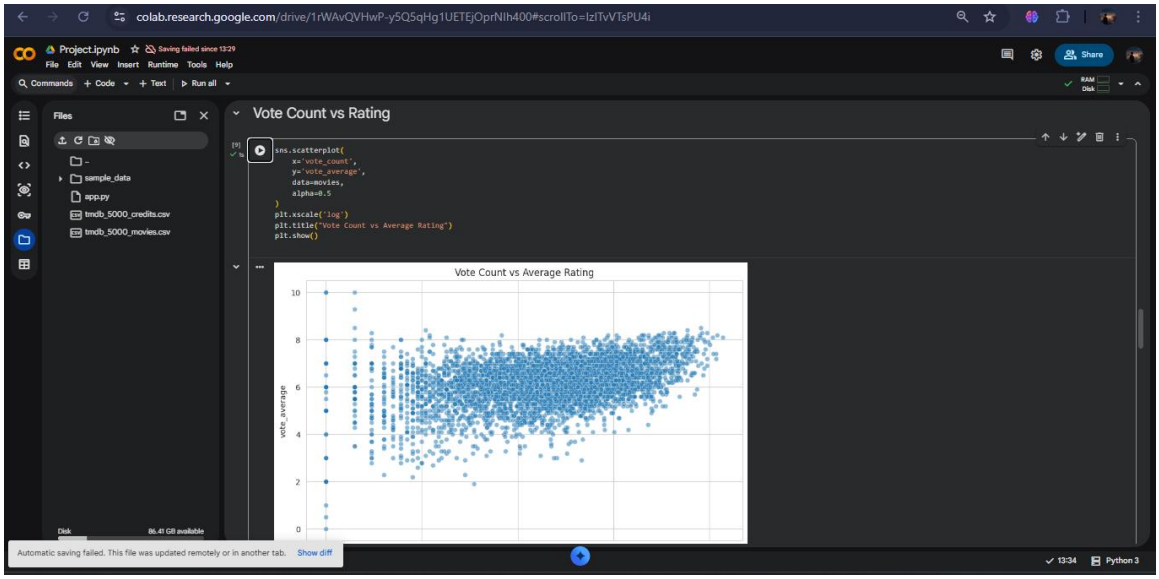
8. Exploratory Data Analysis (EDA)

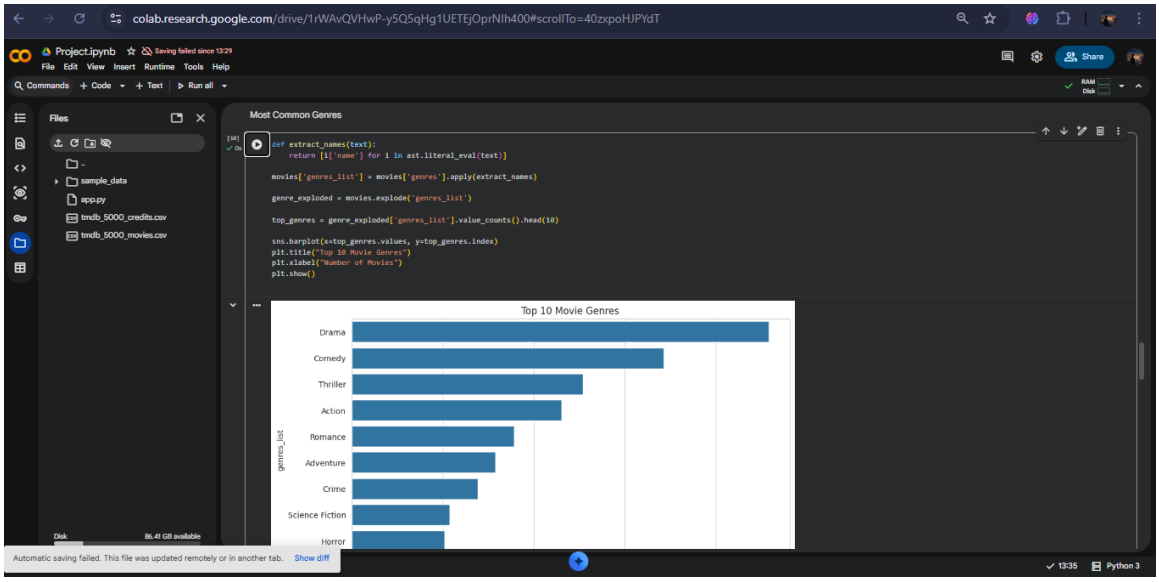
Exploratory Data Analysis (EDA) was performed to gain insights into the dataset and understand underlying patterns. Various visualizations were created to analyze the distribution of movie ratings, vote counts, and popularity.

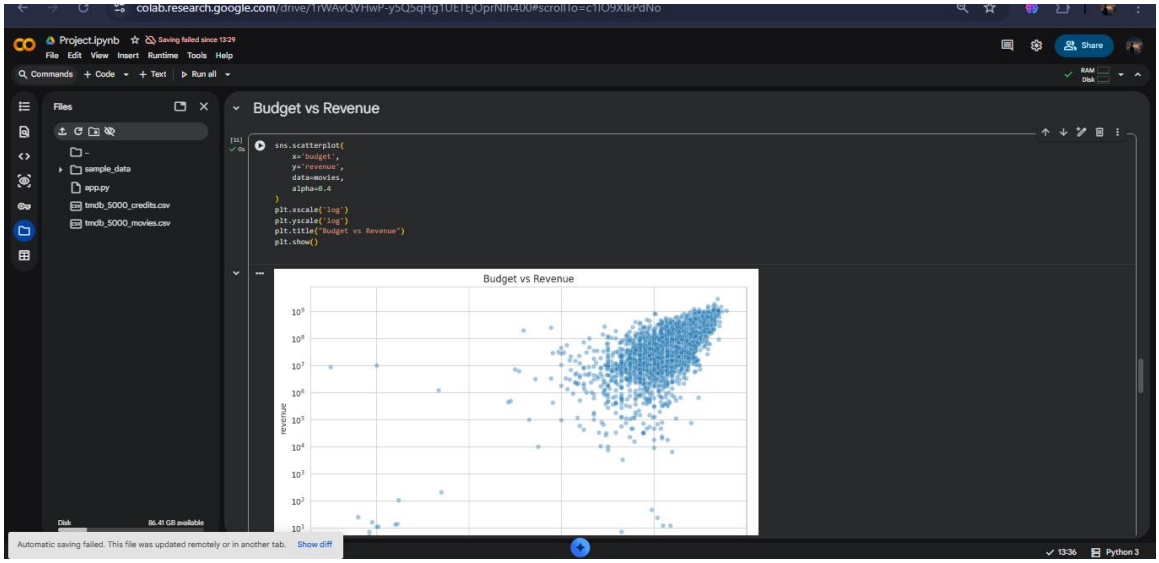
Genre analysis revealed that Drama, Comedy, and Action are the most common genres in the dataset. Scatter plots were used to study the relationship between vote count and average rating, highlighting the presence of rating bias for movies with fewer votes.

These insights helped guide feature engineering and model design decisions.


Project.ipynb
Data Analysis
Rating DistributionRating Distribution
sns.histplot(movies['vote_average'], bins=20, kde=True)
plt.title("Distribution of Movie Ratings")
plt.xlabel("Average Rating")
plt.ylabel("Count")
plt.show()
Distribution of Movie Ratings
Count
1000
800
600
400
200
Files
sample_data
app.py
tmdb_5000_credits.csv
tmdb_5000_movies.csv
Variables
Terminal
Python 3


Project.ipynb
Vote Count vs Rating
sns.scatterplot(
x='vote_count',
y='vote_average',
data=movies,
alpha=0.5
)
plt.xscale('log')
plt.title("Vote Count vs Average Rating")
plt.show()
Vote Count vs Average Rating
vote_average
10
8
6
4
2
0
Automatic saving failed. This file was updated remotely or in another tab.
Show diff
Python 3





9. Feature Engineering

Feature engineering plays a crucial role in improving the performance of the recommendation system. In this project, important features such as genres, keywords, cast, director, and overview were extracted and combined into a single textual feature called 'tags'.

This unified feature captures the semantic information of each movie and serves as the input for the recommendation model. TF-IDF vectorization was applied to convert the text data into numerical form, allowing similarity computation between movies.

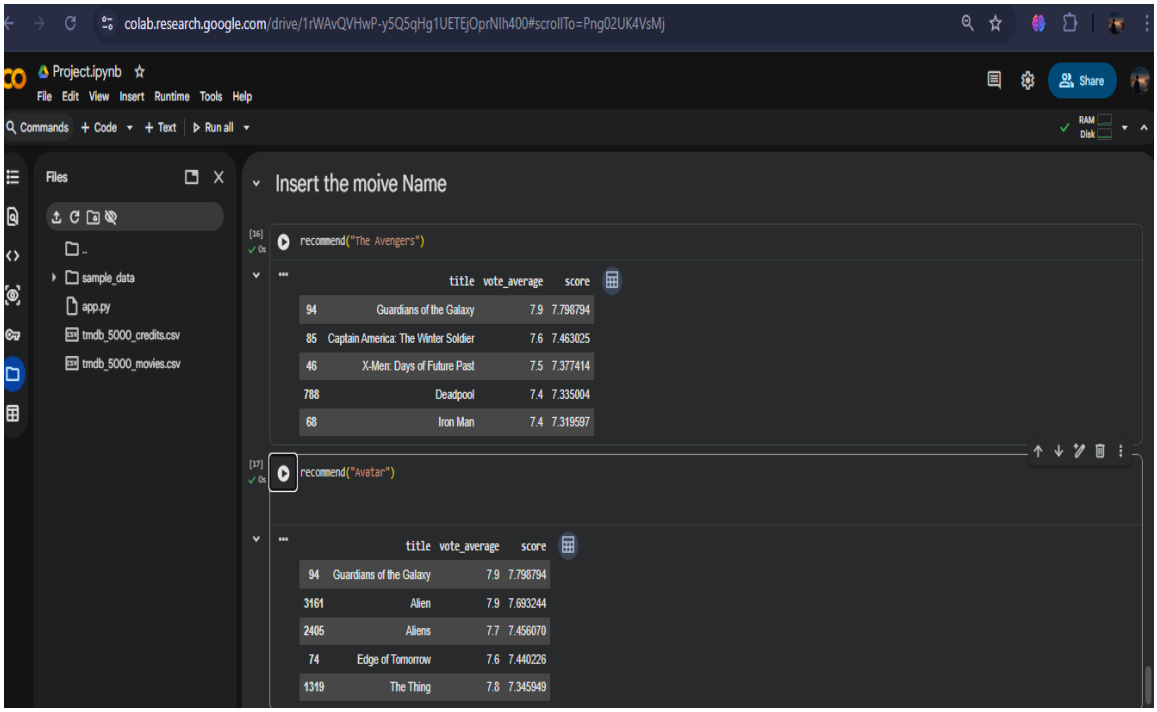
10. Model Building

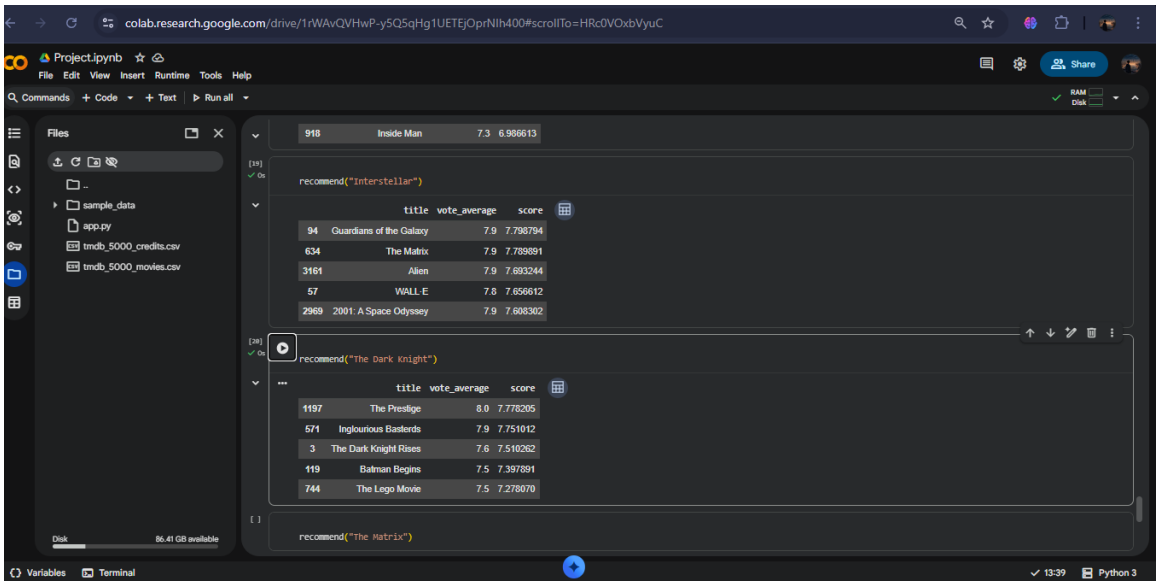
Unsupervised Learning

Unsupervised learning is a type of machine learning in which the model learns patterns and structures from data **without using labeled outputs**. The algorithm analyzes the inherent relationships within the data to identify similarities, clusters, or associations. It is commonly used in applications such as clustering, dimensionality reduction, and recommendation systems. In this project, unsupervised learning is applied to discover similarities between movies based on their content and metadata, enabling effective movie recommendations without predefined target labels.

The recommendation model was built using a content-based filtering approach. TF-IDF vectorization was used to represent movies as numerical vectors based on their content. Cosine similarity was then computed between these vectors to measure the similarity between movies.

This approach was chosen because it is effective for text-based similarity analysis and does not require user interaction data. The model is capable of recommending movies with similar themes, genres, and creators.



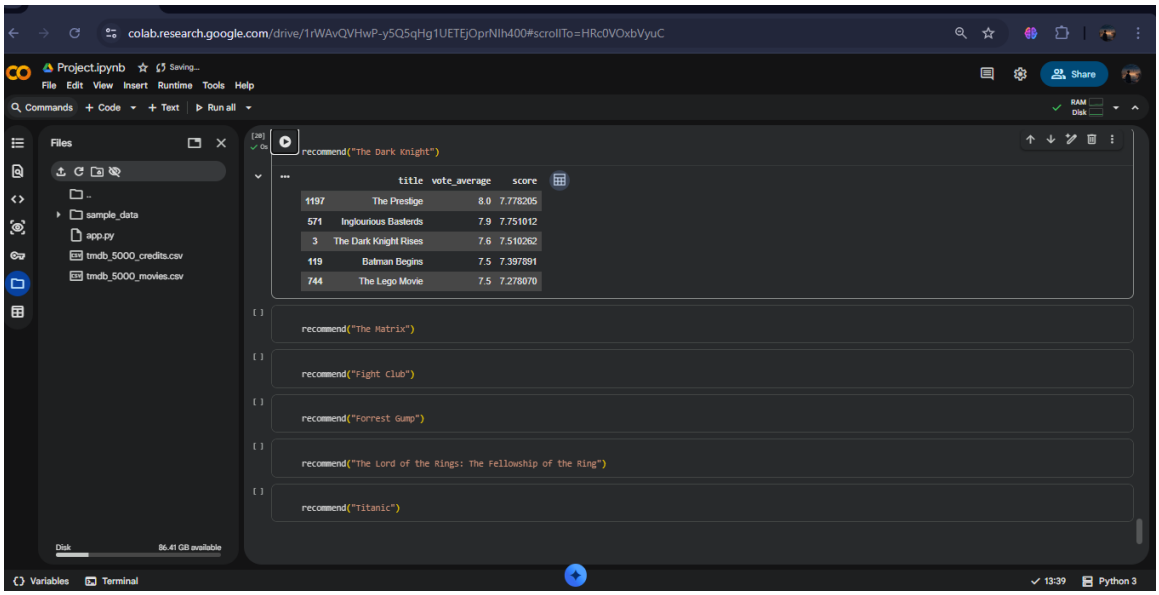


11. Model Evaluation

Evaluating recommendation systems differs from traditional machine learning models. Instead of accuracy or precision, the system was evaluated based on the relevance and quality of recommendations.

The recommendations were manually inspected to ensure that suggested movies were similar in genre, theme, and storyline. A weighted rating mechanism was also used to prioritize higher-quality movies and reduce popularity bias.

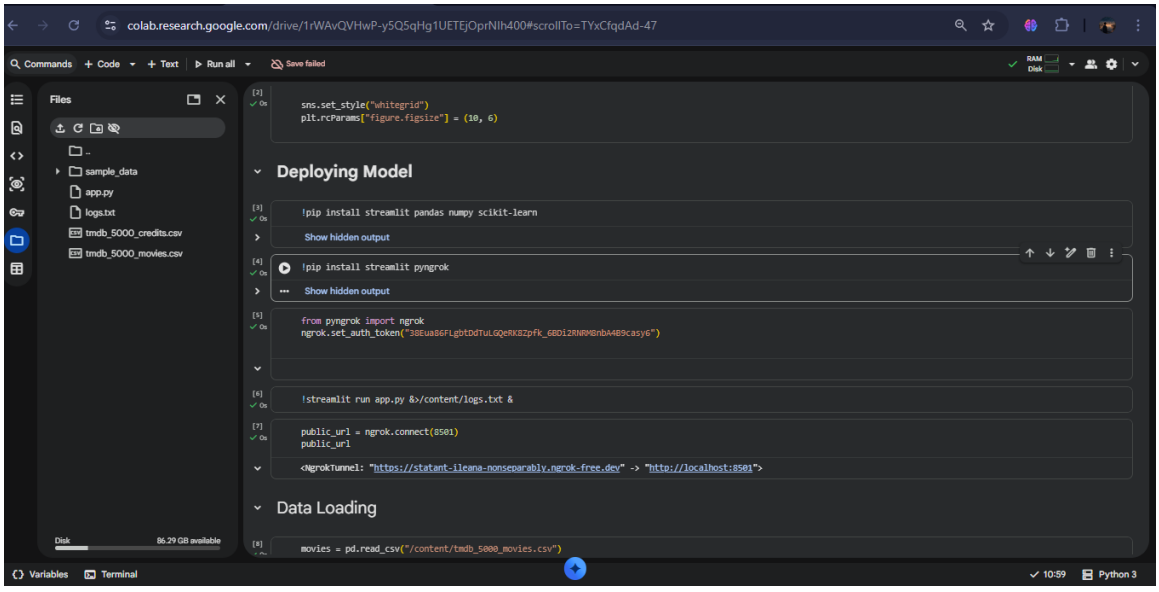
	title	vote_average	score
1197	The Prestige	8.0	7.778205
571	Inglourious Basterds	7.9	7.751012
3	The Dark Knight Rises	7.6	7.510262
119	Batman Begins	7.5	7.397891
744	The Lego Movie	7.5	7.278070

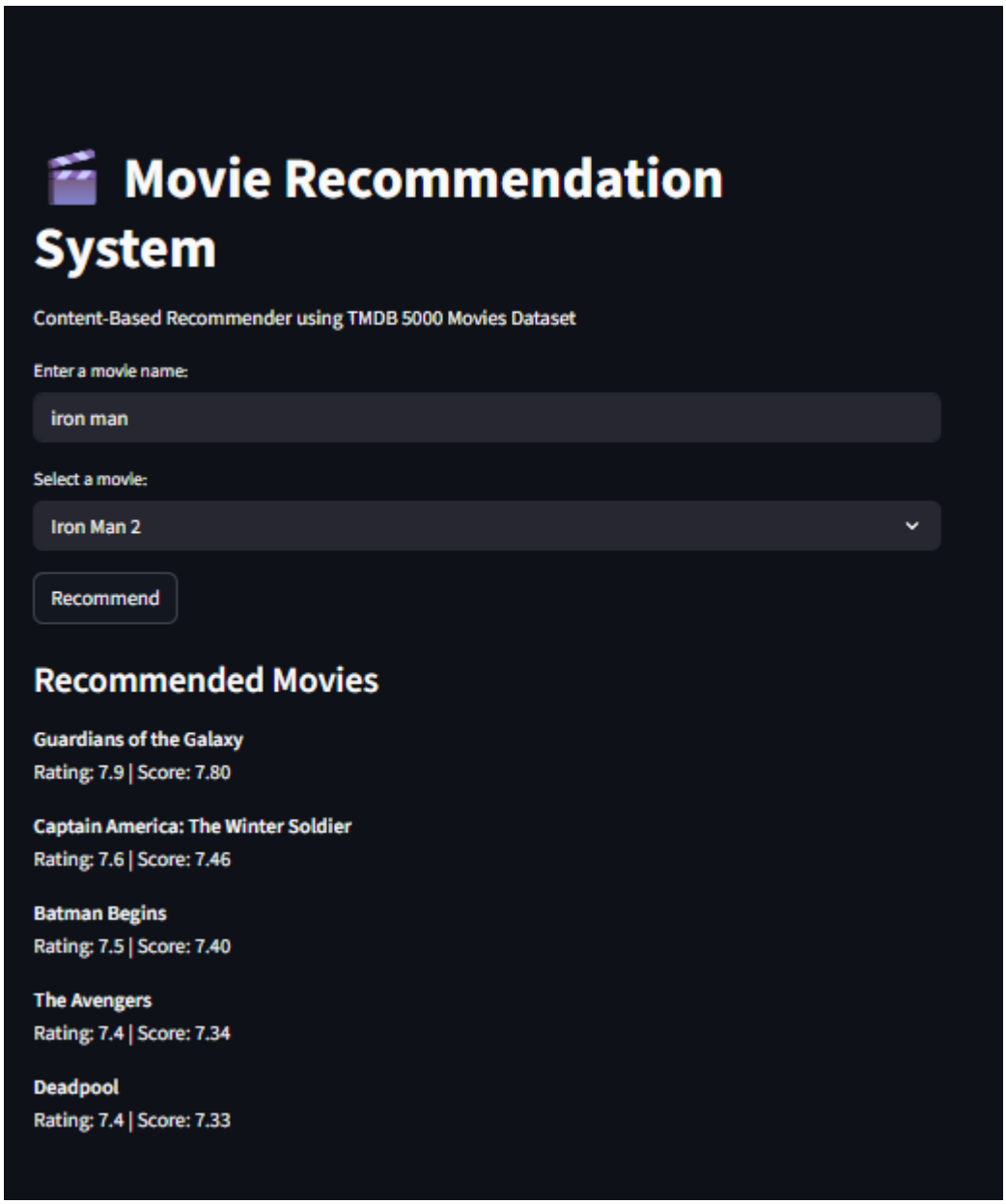


12. Deployment

The final recommendation system was deployed as a web application using Streamlit. The application allows users to enter a movie name, receive auto-suggestions, and view recommended movies instantly.

The deployment demonstrates how machine learning models can be integrated into user-facing applications. The Streamlit interface provides an intuitive and interactive experience.





13. Source Code

The complete source code for this project includes data preprocessing scripts, exploratory data analysis notebooks, feature engineering logic, model building code, and the Streamlit application file (app.py).

App.py

```
import streamlit as st

import pandas as pd

import ast


from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics.pairwise import cosine_similarity


# ----------------------------------------------------

# Streamlit Page Config

# ----------------------------------------------------

st.set_page_config(

   page_title="Movie Recommendation System",

   layout="centered"

)


st.title("🎬 Movie Recommendation System")

st.write("Content-Based Recommender using TMDB 5000 Movies Dataset")


# ----------------------------------------------------

# Load Data (Cached)

# ----------------------------------------------------

@st.cache_data

def load_data():

   movies = pd.read_csv("tmdb_5000_movies.csv")

   credits = pd.read_csv("tmdb_5000_credits.csv")
```

```
    movies = movies.merge(credits, on="title")
    return movies

movies = load_data()

# ------------------------------------------------
# Helper Functions
# ------------------------------------------------
def extract_names(text):
    try:
        return [i["name"] for i in ast.literal_eval(text)]
    except:
        return []

def get_top_cast(text):
    try:
        return [i["name"] for i in ast.literal_eval(text)[:3]]
    except:
        return []

def get_director(text):
    try:
        for i in ast.literal_eval(text):
            if i["job"] == "Director":
                return i["name"]
        return ""
```

```
    except:
        return ""

# ------------------------------------------------
# Feature Engineering
# ------------------------------------------------
movies["overview"] = movies["overview"].fillna("")

movies["genres_list"] = movies["genres"].apply(extract_names)
movies["keywords"] = movies["keywords"].apply(extract_names)
movies["cast"] = movies["cast"].apply(get_top_cast)
movies["director"] = movies["crew"].apply(get_director)

# Normalize text
movies["overview"] = movies["overview"].str.lower()

for col in ["genres_list", "keywords", "cast"]:
    movies[col] = movies[col].apply(
        lambda x: [i.replace(" ", "") for i in x] if isinstance(x, list) else []
    )

movies["director"] = movies["director"].fillna("").replace(" ", "")

# Combine into tags
movies["tags"] = (
    movies["overview"] + " " +
```

```
    movies["genres_list"].apply(lambda x: " ".join(x)) + " " +
    movies["keywords"].apply(lambda x: " ".join(x)) + " " +
    movies["cast"].apply(lambda x: " ".join(x)) + " " +
    movies["director"]
)

# ----------------------------------------------------
# ✅ SAFE final_df CREATION (MANDATORY FIX)
# ----------------------------------------------------
final_df = movies[["title", "tags", "vote_average", "vote_count"]].copy()

final_df["tags"] = final_df["tags"].fillna("")
final_df = final_df[final_df["tags"].str.strip() != ""]
final_df.reset_index(drop=True, inplace=True)

# ----------------------------------------------------
# TF-IDF Vectorization
# ----------------------------------------------------
tfidf = TfidfVectorizer(
    max_features=6000,
    stop_words="english"
)

vectors = tfidf.fit_transform(final_df["tags"])
similarity = cosine_similarity(vectors)
```

```
# ------------------------------------------------
# Weighted Rating (IMDb Formula)
# ------------------------------------------------
C = final_df["vote_average"].mean()
m = final_df["vote_count"].quantile(0.7)

def weighted_rating(x):
    v = x["vote_count"]
    R = x["vote_average"]
    return (v / (v + m) * R) + (m / (m + v) * C)

final_df["score"] = final_df.apply(weighted_rating, axis=1)

# ------------------------------------------------
# Recommendation Function (Case-Insensitive)
# ------------------------------------------------
def recommend(movie_name, top_n=5):
    movie_name = movie_name.lower()
    titles = final_df["title"].str.lower()

    if movie_name not in titles.values:
        return None

    idx = final_df[titles == movie_name].index[0]

    similarity_scores = list(enumerate(similarity[idx]))
```

```
    similarity_scores = sorted(
        similarity_scores,
        key=lambda x: x[1],
        reverse=True
    )[1:50]

    candidates = final_df.iloc[[i[0] for i in similarity_scores]]
    candidates = candidates.sort_values(by="score", ascending=False)

    return candidates[["title", "vote_average", "score"]].head(top_n)

# ------------------------------------------------
# UI
# ------------------------------------------------
movie_input = st.text_input("Enter a movie name:")

if movie_input:
    suggestions = final_df[
        final_df["title"].str.lower().str.contains(movie_input.lower())
    ]["title"].head(10)

    selected_movie = st.selectbox(
        "Select a movie:",
        suggestions if not suggestions.empty else []
    )
```

```
        f"Rating: {row['vote_average']} | "
        f"Score: {row['score']:.2f}"
    )
```

All source code files are maintained in a GitHub repository for version control, collaboration, and reproducibility.

14. Future Scope

There are several opportunities to enhance this project in the future. Collaborative filtering techniques can be integrated to incorporate user behavior and preferences. The TMDB API can be used to fetch real-time movie data, posters, and trailers.

Advanced machine learning and deep learning models such as word embeddings or neural recommendation systems can further improve recommendation quality. Scalability and performance optimizations can also be explored for large-scale deployment.

15. Team Members and Roles

Abhishek KR was responsible for the end-to-end execution of the project. This includes data collection, preprocessing, exploratory data analysis, feature engineering, model building, deployment, and documentation.